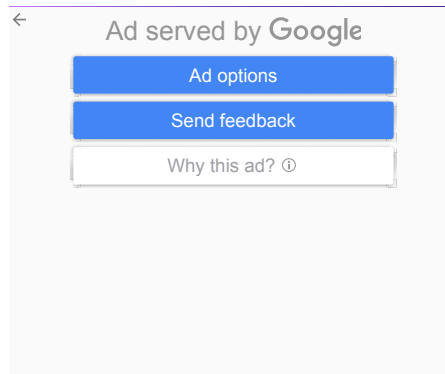




Serving Static Resources in Node.js

In this section, you will learn how to serve static resources like images, css, JavaScript or other static files using **Express.js** and **node-static** module.



Serve Static Resources using Express.js

It is easy to serve static files using built-in middleware in Express.js called `express.static`. Using `express.static()` method, you can server static resources directly by specifying the folder name where you have stored your static resources.

The following example serves static resources from the public folder under the root folder of your application.

server.js

Copy

```
var express = require('express');
var app = express();

//setting middleware
app.use(express.static(__dirname + 'public')); //Serves resources from public
folder

var server = app.listen(5000);
```

In the above example, `app.use()` method mounts the middleware `express.static` for every request. The `express.static` middleware is responsible for serving the static assets of an Express.js application. The `express.static()` method specifies the folder from which to serve all static resources.



Specify absolute path in `express.static()` by prepending `__dirname`. This will not break your application even if you run the express app from another directory.

Now, run the above code using `node server.js` command and point your browser to `http://localhost:5000/myImage.jpg` and it will display myImage.jpg from the public folder (public folder should have myImage.jpg).

If you have different folders for different types of resources then you can set `express.static` middleware as shown below.

Example: Serve resources from different folders

 Copy

```
var express = require('express');
var app = express();

app.use(express.static('public'));

//Serves all the request which includes /images in the url from Images folder
app.use('/images', express.static(__dirname + '/Images'));

var server = app.listen(5000);
```

In the above example, `app.use()` method mounts the `express.static` middleware for every request that starts with `/images`. It will serve images from images folder for every HTTP requests that starts with `/images`. For example, HTTP request `http://localhost:5000/images/myImage.png` will get myImage.png as a response. All other resources will be served from public folder.

Now, run the above code using `node server.js` and point your browser to `http://localhost:5000/images/myImage.jpg` and it will display myImage.jpg from the **images** folder, whereas `http://localhost:5000/myJSFile.js` request will be served from public folder. (images folder must include myImage.png and public folder must include myJSFile.js)

You can also create a virtual path in case you don't want to show actual folder name in the url.

Example: Setting virtual path

 Copy

```
app.use('/resources', express.static(__dirname + '/images'));
```

So now, you can use `http://localhost:5000/resources/myImage.jpg` to serve all the images instead of `http://localhost:5000/images/myImage.jpg`.

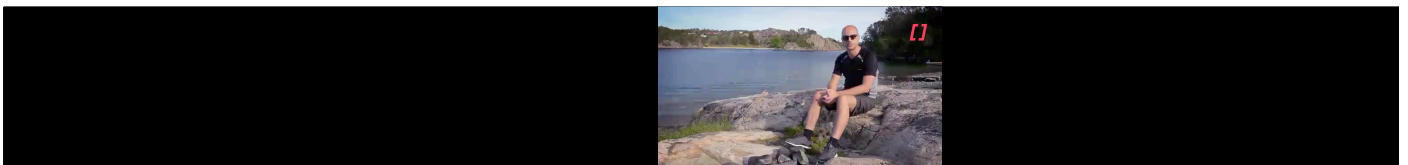
In this way, you can use Express.js to server static resources such as images, CSS, JavaScript or other files.





Serve Static Resources using Node-static Module

In your node application, you can use node-static module to serve static resources. The node-static module is an HTTP static-file server module with built-in caching.



After installing node-static module, you can create static file server in Node.js which serves static files only.

The following example demonstrates serving static resources using node-static module.

Example: Serving static resources using node-static

 Copy

```
var http = require('http');

var nStatic = require('node-static');

var fileServer = new nStatic.Server('./public');

http.createServer(function (req, res) {

    fileServer.serve(req, res);

}).listen(5000);
```

In the above example, node-static will serve static files from public folder by default. So, an URL request will automatically map to the file in the public folder and will send it as a response.

Now, run the above example using `node server.js` command and point your browser to `http://localhost:5000/myImage.jpg` (assuming that public folder includes myImage.jpg file) and it will display the image on your browser. You don't need to give `/public/myImage.jpg` because it will automatically serve all the static files from the public folder.

Visit [Github](#) to learn about node-static in detail.

Want to check how much you know Node.js?